

A Course Based Project on
Submitted to the Department of ECE

AUTOMATED TRAFFIC SIGNAL SYSTEM

in partial fulfillment of the requirements for the completion on course

ANALOG CIRCUITS LABORATORY (22PC2EC205)

BACHELOR OF TECHNOLOGY
in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Y. Venkata Mokshagna	24071A04K8
P.Abhinash	25075A0417
R. Naveen Kumar	25075A0418
R. Suma	25075A0419

UNDER THE SUPERVISION OF

Dr. K. Aruna Kumari, Assistant Professor

Mrs.Y. Sri Rama, Assistant Professor



VNRVJIET

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY
AUTONOMOUS INSTITUTION

A.Y. 2025-26

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Electronics and Communication Engineering



CERTIFICATE

This is to certify that the course based project entitled “**AUTOMATED TRAFFIC SIGNAL SYSTEM**” is being submitted by **Y. Venkata Mokshagna (24071A04K8)**, **P.Abhinash(25075A0417)**, **R.Naveen(25075A0418)**, **R.Suma(25075A0419)** for the partial fulfillment of requirements for course based project of **Analog Circuits Laboratory** in II year II semester of **Bachelor of Technology** in Electronics and Communication Engineering of VNRVJIET, Hyderabad during the academic year 2025 - 2026.

Course coordinator

Dr.K.Aruna Kumari

Assistant professor,ECE
VNRVJIET,Hyd

Course coordinator

Mrs.Y.Sri Rama

Assistant professor,ECE
VNRVJIET,Hyd

Head of the Department

Dr.P.Kishore

Assoc.Professor & Head
VNRVJIET,Hyd

DECLARATION

We do declare that the course based project report entitled “**AUTOMATED TRAFFIC SIGNAL SYSTEM**” submitted to the Department of Electronics and Communication Engineering, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfillment of the requirements for course based project of [Analog Circuits Laboratory](#) in II B.Tech. II Semester for the academic year 2025 - 2026.

Place: Hyderabad

Date:

Student Name	Roll No.	Student Signature
1. Y. Venkata Mokshagna	24071A04K8	
2. P Abhinash.	25075A0417	
3. R Naveen Kumar	25075A0418	
4. R.Suma	25075A0419	

Verified by:

Dr. K. Aruna Kumari
Assistant Professor, ECE

Mrs. Y. Sri Rama
Assistant Professor, ECE

Date of Verification:

CONTENTS

CERTIFICATE	i
DECLARATION	ii
CONTENTS	iii
ABSTRACT	1
1. INTRODUCTION	2
2. BLOCK DIAGRAM	3
3. WORKING	5
4. OUTPUT	7
5. CONCLUSION	8
6. REFERENCES	9
7.Appendix	10

ABSTRACT

Rapid urbanization and the increasing number of vehicles on roads have led to severe traffic congestion and inefficient traffic management in many cities. Traditional traffic signal systems operate on fixed time intervals, which do not adapt to real-time traffic conditions, resulting in unnecessary delays and fuel wastage. This paper proposes an automated traffic signal system that dynamically controls signal timings based on real-time traffic density.

The system utilizes sensors, cameras, or IoT-enabled devices to monitor vehicle flow at intersections. Data collected is processed using intelligent algorithms to adjust signal durations, giving priority to lanes with higher traffic density. Advanced implementations may incorporate machine learning techniques for predictive traffic management and emergency vehicle prioritization.

The proposed system aims to reduce congestion, minimize waiting time, improve fuel efficiency, and enhance overall road safety. By replacing static traffic control with an adaptive approach, the automated traffic signal system offers a scalable and efficient solution for modern smart cities.

This project presents an automated traffic signal system that adjusts signal timings based on real-time traffic conditions. Unlike conventional fixed-time signals, the system uses sensors or cameras to detect vehicle density at intersections and dynamically allocates green time to reduce congestion. It can also prioritize emergency vehicles when needed. The system improves traffic flow, reduces waiting time and fuel consumption, and enhances road safety, making it suitable for smart city applications

1. INTRODUCTION

Traffic congestion has become a major issue in urban areas due to the rapid increase in the number of vehicles and limited road infrastructure. Conventional traffic signal systems operate on fixed time intervals, which do not consider real-time traffic conditions. This often leads to inefficient traffic flow, longer waiting times, and increased fuel consumption.

An automated traffic signal system is designed to overcome these limitations by adapting signal timings based on current traffic density. The system uses sensors, cameras, or IoT devices to monitor vehicle flow at intersections and processes this data to control traffic lights intelligently. By allocating green signals according to traffic demand, it ensures smoother movement of vehicles and reduces congestion.

Such systems can also be enhanced with advanced technologies like machine learning and image processing to predict traffic patterns and prioritize emergency vehicles. Overall, automated traffic signal systems play a key role in improving road efficiency, reducing environmental impact, and supporting the development of smart cities.

Traffic congestion is a growing problem in urban areas due to the increasing number of vehicles. Traditional traffic signals operate on fixed timings, which often leads to delays and inefficient traffic flow. An automated traffic signal system addresses this issue by adjusting signal timings based on real-time traffic conditions using sensors or cameras. This approach helps reduce congestion, saves time and fuel, and improves overall road safety.

Traffic congestion is a major challenge in modern cities due to the rapid growth in vehicle usage. Traditional traffic signals operate on fixed time intervals, which are not efficient for varying traffic conditions and often cause unnecessary delays. An automated traffic signal system improves this by using sensors, cameras, or IoT devices to monitor real-time traffic at intersections.

Based on the collected data, the system dynamically adjusts signal timings to manage traffic flow more efficiently. It can also give priority to emergency vehicles and reduce waiting times at signals. This leads to lower fuel consumption, reduced air pollution, and improved road safety.

2. BLOCK DIAGRAM

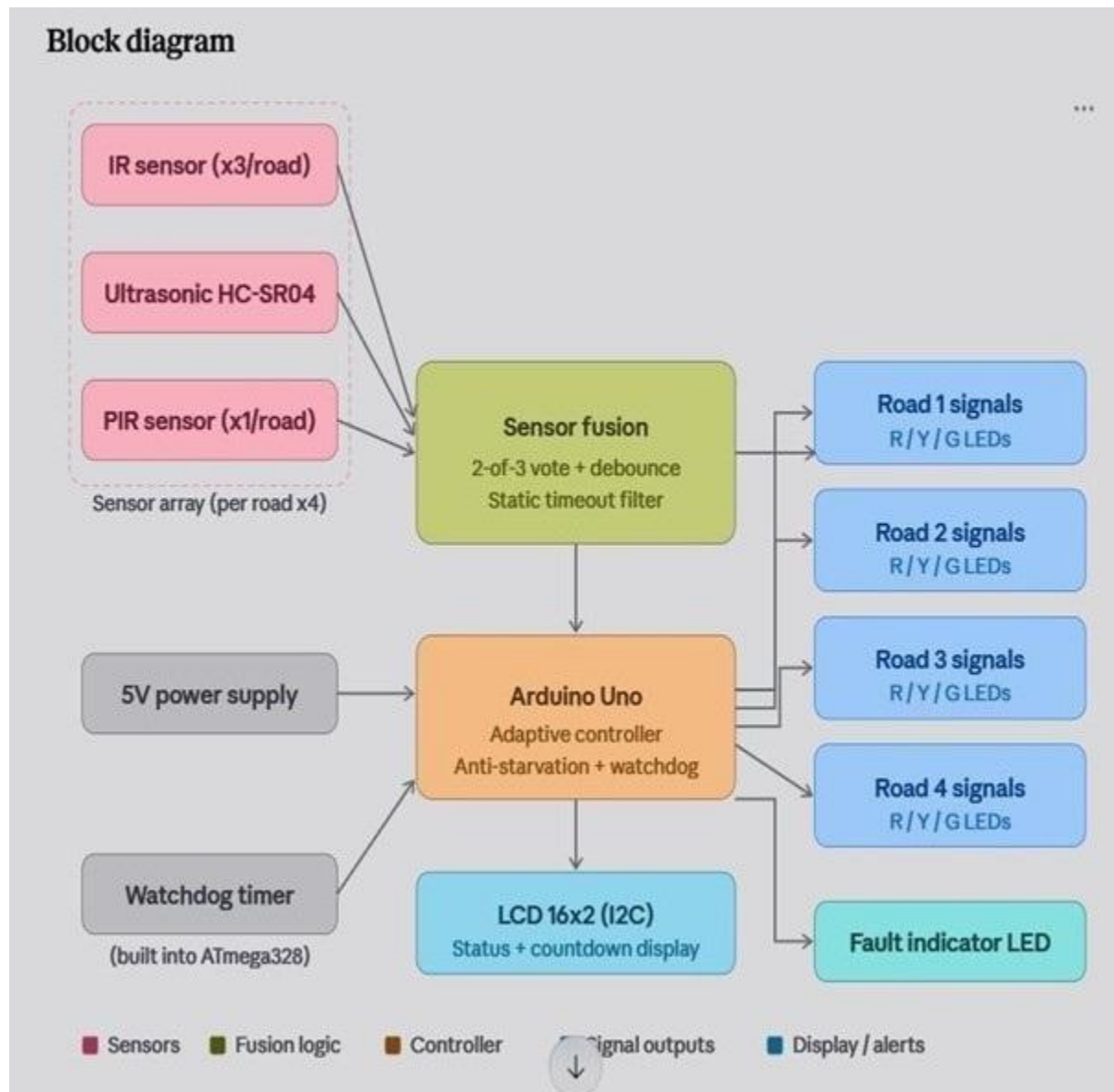


Fig:2.1:Process of Automated Traffic Signal System

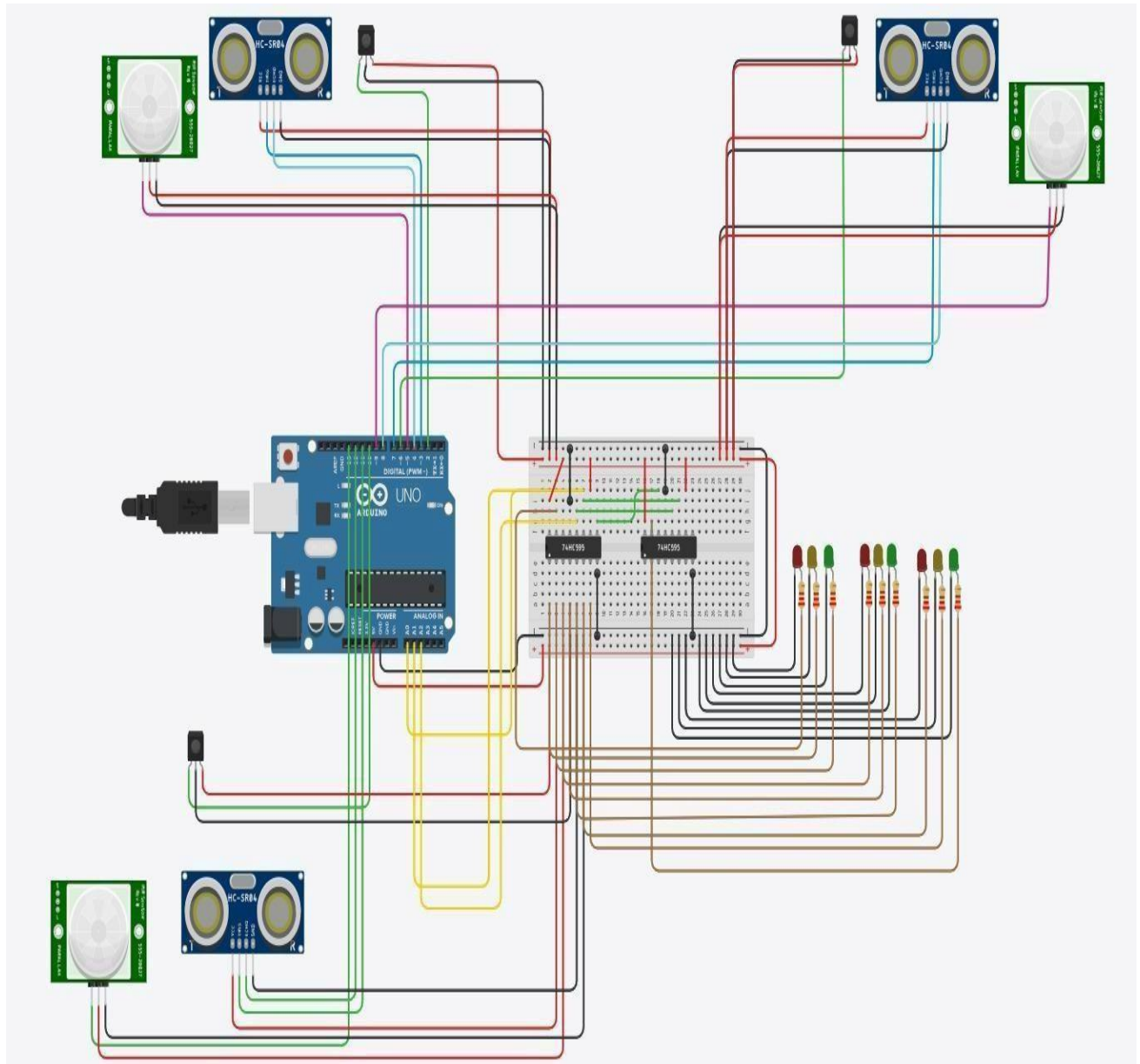


Fig.2.2:Circuit diagram of Automated Traffic Signal System

3. WORKING

The project works as a smart traffic management system in which three roads are continuously monitored using a combination of PIR, ultrasonic, and IR sensors connected to an Arduino Uno. The main objective of the system is to replace fixed-time traffic signals with a dynamic decision-making mechanism that reacts according to actual vehicle presence. Since a single sensor is often unreliable in real-world conditions, the project uses sensor fusion, where multiple sensors cooperate to confirm traffic before the signal timing is changed. The Arduino acts as the central controller, processing all sensor inputs, applying decision logic, and controlling the traffic LEDs through two 74HC595 shift registers, which expand the number of output pins and allow all red, yellow, and green LEDs to be controlled efficiently using fewer Arduino pins.

The working of the system begins with simultaneous sensor sampling. Instead of checking one road at a time, the Arduino first reads all sensors from all roads together and only then makes a decision. This avoids timing mismatch problems where one road could be processed earlier than another, leading to unfair or unstable traffic detection. For every road, the PIR sensor is used as the first stage of detection. PIR detects motion by sensing changes in infrared radiation, so when a vehicle approaches the road, the PIR sensor becomes active and indicates possible traffic movement. However, PIR alone is not reliable because it only detects motion and may stop detecting when the vehicle becomes stationary. Therefore, the system does not immediately confirm traffic from PIR alone. Instead, it creates a temporary “latch” or memory state that remains active for several seconds even after the PIR output disappears. This helps stabilize detection and prevents rapid ON/OFF fluctuations.

After the PIR stage, the ultrasonic sensor acts as the second level of validation. The ultrasonic sensor continuously measures the distance of nearby objects by sending sound waves and measuring the reflected echo time. The code compares the current distance with previous distance values to identify meaningful movement. Ultrasonic detection is allowed only if the PIR latch is already active, meaning that motion must be detected before distance validation is accepted. This gating logic is very important because it prevents false triggers caused by random obstacles or parked vehicles. If the ultrasonic sensor detects a valid distance change after PIR activation, another latch is created for the ultrasonic sensor. If the ultrasonic sensor repeatedly sees an object at almost the same distance for a long period without any PIR activity, the system assumes that the object is parked or static rather than part of moving traffic. In this case, the ultrasonic sensor is marked as faulty or disqualified for that road,

preventing parked vehicles from continuously triggering traffic detection.

The IR sensor acts as the final confirmation stage. Unlike PIR, which detects motion, the IR sensor detects whether a vehicle is physically present near the stop line of the signal. The IR sensor is active LOW, meaning it becomes active when an object blocks its infrared reflection path. Once the IR sensor remains active for a minimum stable duration, the system creates an IR latch that also stays active temporarily even if the signal disappears momentarily. This makes the IR detection stable and resistant to small fluctuations caused by vibration or quick movements. Together, the PIR, ultrasonic, and IR latches create a multi-stage validation pipeline consisting of movement detection, distance confirmation, and final presence verification.

After all three sensor stages are processed, the system applies a voting mechanism. Each active latch contributes one vote. If at least two out of the three sensors agree, the system confirms that traffic is present on that road. This “2-of-3” voting system improves reliability because a single faulty or noisy sensor cannot falsely trigger the signal. For example, PIR and ultrasonic together can confirm an approaching vehicle even before it reaches the IR sensor, while ultrasonic and IR together can confirm a stationary waiting vehicle. If only one sensor remains active, traffic is not immediately removed because the system includes a hold-off mechanism. Once traffic has been confirmed, the traffic state remains active for a few additional seconds even if sensor votes temporarily drop. This prevents unstable flickering where traffic rapidly appears and disappears due to small sensor variations.

Once traffic detection is completed for all roads, the decision-making stage begins. The controller first checks whether any road has active traffic. If no traffic exists, or if all roads show equal traffic conditions, the system enters a default cyclic mode where signals rotate in the fixed order Road 1 → Road 2 → Road 3. This ensures continuous operation even when no smart decision is needed. However, if traffic exists on specific roads, the system enters smart mode. In smart mode, the road that has been waiting in red for the longest time is given priority. This waiting duration is tracked using a timer variable called redSince. The system also implements anti-starvation logic, which ensures that no road remains red for more than a specified maximum duration. If a road has active sensor latches and has been waiting too long, the controller forcibly grants green to that road even if another road still has traffic. This prevents smaller roads from being permanently ignored during heavy traffic conditions.

The signal control phase itself is carefully structured. When a road is selected, its green LED turns ON while all other roads remain red. The minimum green duration is fixed at five seconds to ensure proper traffic flow. If traffic is still detected on the same road after the minimum duration, the system

allows a one-time extension up to ten seconds. This dynamic extension ensures that crowded roads get more green time while still maintaining fairness. After the green phase ends, the signal changes to yellow for three seconds, warning vehicles that the signal is about to stop. Then an all-red phase of two seconds is activated where every road remains red simultaneously. This safety gap prevents collisions during transitions between roads. After this, the selected road's red timer is reset because the road has now physically returned to red status.

The system also includes several reliability and safety mechanisms. One important feature is stuck-sensor detection. If traffic remains continuously active for more than ninety seconds on a road, the system assumes a sensor malfunction. In such a case, the controller raises an alert, clears all sensor latches for that road, disables smart traffic control for that sensor, and returns the road to default-mode operation. This prevents the system from permanently locking onto one road because of a faulty sensor. Additionally, all sensor states, votes, traffic decisions, fault alerts, latch conditions, and timing information are continuously printed to the serial monitor in a structured table format. This helps debugging, monitoring, and demonstration during testing.

Overall, the project demonstrates a complete embedded traffic control system that combines real-time sensing, sensor fusion, stabilization logic, fault handling, priority scheduling, and dynamic signal timing into a single low-cost prototype. Instead of relying on fixed timers or a single unreliable sensor, the system intelligently validates traffic using multiple conditions before making decisions, resulting in a more stable and practical approach to smart traffic management.

4.OUTPUT

The output of the automated traffic signal system is the controlled switching of traffic lights (red, yellow, and green) based on real-time traffic conditions. The system provides optimized signal timings for each lane, ensuring smoother vehicle movement at intersections.

As a result, it produces reduced traffic congestion, shorter waiting times, improved traffic flow, and lower fuel consumption. In advanced systems, the output may also include priority signals for emergency vehicles and better coordination between multiple intersections.

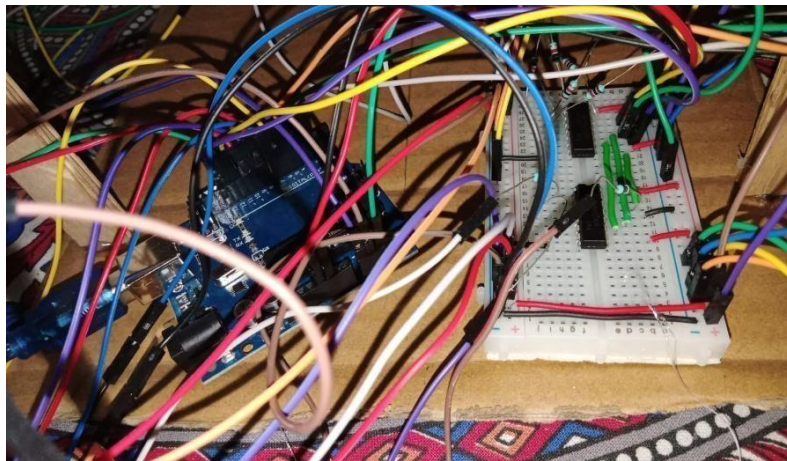


Fig.4.1: Output circuit diagram

4. CONCLUSION

In conclusion, this project successfully demonstrates the design and implementation of a smart traffic control system using sensor fusion and dynamic decision-making. By combining PIR, ultrasonic, and IR sensors, the system improves the reliability of vehicle detection compared to traditional single-sensor or fixed-timer approaches. The implemented latch mechanism, voting logic, hold-off timing, anti-starvation control, and fault detection make the system stable and capable of handling real-world sensor inconsistencies such as noise, temporary signal loss, and parked vehicles. The controller dynamically allocates signal timing based on actual traffic conditions while still maintaining a safe default cyclic mode when traffic is absent or equal across roads. Although the current setup is a prototype with limited coverage and simplified traffic estimation, it provides a strong foundation for future expansion into larger multi-lane and multi-point intelligent traffic systems. Overall, the project achieves its objective of creating a low-cost, scalable, and practical smart traffic management solution using embedded systems and sensor fusion techniques.

5. REFERENCES

1. S. K. Sharma and P. Singh, *Smart Traffic Management Systems*, IEEE Conference Proceedings, 2022.
2. N. Gupta and R. Verma, “Intelligent Traffic Control Using IoT,” *International Journal of Engineering Research*, 2021.
3. A. K. Jain, *Traffic Engineering and Transport Planning*, Khanna Publishers.
4. IEEE Xplore Digital Library – Research papers on Adaptive Traffic Signal Control Systems.
5. G. Patel et al., “Real-Time Traffic Monitoring and Control Using Sensors,” *International Journal of Computer Applications*, 2020.

7.APPENDIX

7.1.Code:

```
#include <Arduino.h>

#include <avr/wdt.h>

static const uint8_t R1_PIR=2, R1_TRIG=3, R1_ECHO=4, R1_IR=5;
static const uint8_t R2_PIR=6, R2_TRIG=7, R2_ECHO=8, R2_IR=9;
static const uint8_t R3_PIR=10, R3_TRIG=A0, R3_ECHO=12, R3_IR=A1;
static const uint8_t SR_DATA=11, SR_CLK=13, SR_LATCH=A2;
static const uint16_t B_R1_RED=(1<<0), B_R1_YEL=(1<<1), B_R1_GRN=(1<<2);
static const uint16_t B_R2_RED=(1<<3), B_R2_YEL=(1<<4), B_R2_GRN=(1<<5);
static const uint16_t B_R3_RED=(1<<8), B_R3_YEL=(1<<9), B_R3_GRN=(1<<10);
static const uint16_t ALL_RED_BITS = B_R1_RED | B_R2_RED | B_R3_RED;
static const uint32_t MIN_GREEN_MS    = 5000UL;
static const uint32_t GREEN_CAP_MS    = 10000UL;
static const uint32_t YELLOW_MS      = 3000UL;
static const uint32_t ALL_RED_MS      = 2000UL;
static const uint32_t DEFAULT_GREEN_MS = 5000UL;
static const uint32_t ANTI_STARVE_MS  = 25000UL;
static const uint32_t SENSOR_STUCK_MS = 90000UL;
static const uint32_t PIR_LATCH_MS    = 8000UL;
static const uint32_t US_LATCH_MS     = 6000UL;
static const uint32_t IR_LATCH_MS     = 5000UL;
static const uint32_t PIR_MIN_ACTIVE_MS = 300UL;
static const uint32_t US_MIN_ACTIVE_MS = 500UL;
static const uint32_t IR_MIN_ACTIVE_MS = 400UL;
static const uint32_t TRAFFIC_HOLDOFF_MS = 4000UL;
static const uint32_t STATIC_TIMEOUT_MS = 25000UL;
static const uint32_t LOG_INTERVAL_MS = 500UL;
static const uint16_t US_MAX_DIST_CM  = 350;
static const uint8_t US_TRIGGER_CHANGE = 10;
static const uint8_t US_CHANGE_CM     = 8;
static const uint8_t PIR_PINS[3] = {R1_PIR, R2_PIR, R3_PIR };
static const uint8_t TRIG_PINS[3] = {R1_TRIG, R2_TRIG, R3_TRIG};
static const uint8_t ECHO_PINS[3] = {R1_ECHO, R2_ECHO, R3_ECHO};
```

```

static const uint8_t IR_PINS[3] = {R1_IR, R2_IR, R3_IR };

struct Road {
    bool   pirRaw;
    bool   irRaw;
    int32_t usDist;
    int32_t prevUsDist;
    uint32_t pirActiveStart;
    bool   pirConfirmed;
    uint32_t pirLatchExpiry;
    bool   pirLatch;
    uint32_t usActiveStart;
    bool   usConfirmed;
    uint32_t usLatchExpiry;
    bool   usLatch;
    int32_t lastUsDist;
    uint32_t staticSince;
    bool   usFault;
    uint32_t irActiveStart;
    bool   irConfirmed;
    uint32_t irLatchExpiry;
    bool   irLatch;
    int    votes;
    bool   trafficPresent;
    uint32_t lastVoteDropTime;
    bool   inHoldoff;
    uint32_t trafficOnSince;
    bool   sensorFault;
    bool   prevTraffic;
    int    prevVotes;
    uint32_t lastGreenAt;
    uint32_t redSince;
};

static Road   road[3];

static int8_t activeRoad = -1;

static uint8_t defaultRoad = 0;

static void writeLEDs(uint16_t pattern) {

```



```

    digitalWrite(SR_LATCH, LOW);
    shiftOut(SR_DATA, SR_CLK, MSBFIRST,
        (uint8_t)((pattern >> 8) & 0xFF));
    shiftOut(SR_DATA, SR_CLK, MSBFIRST,
        (uint8_t)(pattern & 0xFF));
    digitalWrite(SR_LATCH, HIGH);
}

static void setAllRed() {
    writeLEDs(ALL_RED_BITS);
}

static uint16_t buildPattern(uint8_t r, bool yellow) {
    uint16_t p = ALL_RED_BITS;
    if (r == 0) {
        p &= ~B_R1_RED;
        p |= yellow ? B_R1_YEL : B_R1_GRN;
    }
    if (r == 1) {
        p &= ~B_R2_RED;
        p |= yellow ? B_R2_YEL : B_R2_GRN;
    }
    if (r == 2) {
        p &= ~B_R3_RED;
        p |= yellow ? B_R3_YEL : B_R3_GRN;
    }
    return p;
}

static int32_t readUS(uint8_t trig, uint8_t echo) {
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);
    long d = pulseIn(echo, HIGH, 25000UL);
    return d ? (int32_t)(d * 0.034f / 2.0f) : 999;
}

static void snapshotAllSensors() {

```

```

for (uint8_t r = 0; r < 3; r++) {
    road[r].pirRaw = (digitalRead(PIR_PINS[r]) == HIGH);
    road[r].usDist = readUS(TRIG_PINS[r], ECHO_PINS[r]);
    road[r].irRaw = (digitalRead(IR_PINS[r]) == LOW);
    wdt_reset();
}
}

static void printLine() {
    Serial.println(
        F("----+-----+-----+-----+-----+-----+-----+"));
}

static void printTableHeader() {
    printLine();
    Serial.println(
        F("| Rd | PIR r/L | US cm/St | IR r/L | Votes | Traf | Red(s) |"));
    printLine();
}

static void printRoadRow(uint8_t r) {
    Road& rd = road[r];
    Serial.print(F("| R"));
    Serial.print(r + 1);
    Serial.print(F(" | "));
    Serial.print(rd.pirRaw ? F("Y") : F("n"));
    Serial.print(F("/"));
    Serial.print(rd.pirLatch ? F("LATCH ") : F(""));
    Serial.print(F(" | "));
    char distBuf[5];
    if (rd.usDist == 999) {
        snprintf(distBuf, sizeof(distBuf), "---");
    } else {
        snprintf(distBuf, sizeof(distBuf), "%3ld", (long)rd.usDist);
    }
    Serial.print(distBuf);
    Serial.print(F("cm/"));
    if (rd.sensorFault) Serial.print(F("FLT "));
    else if (rd.usFault) Serial.print(F("PRK "));

```

```

else if (rd.usLatch) Serial.print(F("L "));
else Serial.print(F(" "));
Serial.print(F(" | "));
Serial.print(rd.irRaw ? F("Y") : F("n"));
Serial.print(F("/"));
Serial.print(rd.irLatch ? F("LATCH ") : F(" "));
Serial.print(F(" | "));
Serial.print(F(" "));
Serial.print(rd.votes);
Serial.print(F(" | "));
if (rd.sensorFault) Serial.print(F("FAULT "));
else if (rd.trafficPresent) Serial.print(F("YES "));
else if (rd.inHoldoff) Serial.print(F("HOLD "));
else Serial.print(F("no "));
Serial.print(F(" | "));
uint32_t s = (millis() - rd.redSince) / 1000UL;
char secBuf[5];
snprintf(secBuf, sizeof(secBuf), "%4lu", (unsigned long)s);
Serial.print(secBuf);
Serial.println(F("s |"));
}

static void printStatusTable() {
    printTableHeader();
    for (uint8_t r = 0; r < 3; r++) {
        printRoadRow(r);
    }
    printLine();
}

static void processFusion(uint8_t r) {
    uint32_t now = millis();
    bool newPir = road[r].pirRaw;
    int32_t newDist = road[r].usDist;
    bool newIr = road[r].irRaw;
    if (newPir) {
        if (!road[r].pirActiveStart)
            road[r].pirActiveStart = now;
    }

```

```

    if (!road[r].pirConfirmed &&
        (now - road[r].pirActiveStart) >= PIR_MIN_ACTIVE_MS) {
        road[r].pirConfirmed = true;
    }
    if (road[r].pirConfirmed) {
        road[r].pirLatchExpiry = now + PIR_LATCH_MS;
        road[r].pirLatch = true;
    }
} else {
    road[r].pirActiveStart = 0;
    road[r].pirConfirmed = false;
}
if (road[r].pirLatch && now >= road[r].pirLatchExpiry) {
    road[r].pirLatch = false;
}
bool distChanged =
    (abs(newDist - road[r].lastUsDist) >= US_CHANGE_CM);
road[r].lastUsDist = newDist;
if (newDist > 0 && newDist < US_MAX_DIST_CM &&
    !newPir && !distChanged) {
    if (!road[r].staticSince)
        road[r].staticSince = now;
    if (!road[r].usFault &&
        (now - road[r].staticSince) >= STATIC_TIMEOUT_MS) {
        road[r].usFault = true;
    }
} else {
    road[r].usFault = false;
    road[r].staticSince = 0;
}
bool usTriggered = false;
if (!road[r].usFault &&
    newDist > 0 &&
    newDist < US_MAX_DIST_CM) {
    int32_t prev = road[r].prevUsDist;
    if (prev == 999 ||

```

```

    abs(prev - newDist) >= US_TRIGGER_CHANGE) {
        usTriggered = true;
    }
    road[r].prevUsDist = newDist;
} else {
    road[r].prevUsDist = newDist;
}
if (usTriggered && road[r].pirLatch) {
    if (!road[r].usActiveStart)
        road[r].usActiveStart = now;
    if (!road[r].usConfirmed &&
        (now - road[r].usActiveStart) >= US_MIN_ACTIVE_MS) {
        road[r].usConfirmed = true;
    }
    if (road[r].usConfirmed) {
        road[r].usLatchExpiry = now + US_LATCH_MS;
        road[r].usLatch = true;
    }
} else if (!usTriggered) {
    road[r].usActiveStart = 0;
    road[r].usConfirmed = false;
}
if (road[r].usLatch && now >= road[r].usLatchExpiry) {
    road[r].usLatch = false;
}
if (newIr) {
    if (!road[r].irActiveStart)
        road[r].irActiveStart = now;
    if (!road[r].irConfirmed &&
        (now - road[r].irActiveStart) >= IR_MIN_ACTIVE_MS) {
        road[r].irConfirmed = true;
    }
    if (road[r].irConfirmed) {
        road[r].irLatchExpiry = now + IR_LATCH_MS;
        road[r].irLatch = true;
    }
}

```

```

} else {
    road[r].irActiveStart = 0;
    road[r].irConfirmed = false;
}
if (road[r].irLatch && now >= road[r].irLatchExpiry) {
    road[r].irLatch = false;
}
int v = 0;
if (road[r].pirLatch) v++;
if (road[r].usLatch && !road[r].usFault) v++;
if (road[r].irLatch) v++;
road[r].votes = v;
if (v >= 2 && !road[r].sensorFault) {
    road[r].trafficPresent = true;
    road[r].inHoldoff = false;
    road[r].lastVoteDropTime = 0;
    if (!road[r].trafficOnSince)
        road[r].trafficOnSince = now;
} else {
    if (road[r].trafficPresent) {
        if (!road[r].inHoldoff) {
            road[r].inHoldoff = true;
            road[r].lastVoteDropTime = now;
        }
        if ((now - road[r].lastVoteDropTime)
            < TRAFFIC_HOLDOFF_MS) {
            road[r].trafficPresent = true;
        } else {
            road[r].trafficPresent = false;
            road[r].inHoldoff = false;
            road[r].lastVoteDropTime = 0;
            road[r].trafficOnSince = 0;
        }
    }
}
}
if (road[r].trafficPresent &&

```

```

road[r].trafficOnSince) {
if (!road[r].sensorFault &&
(now - road[r].trafficOnSince)
>= SENSOR_STUCK_MS) {
road[r].sensorFault = true;
road[r].trafficPresent = false;
road[r].pirLatch = false;
road[r].usLatch = false;
road[r].irLatch = false;
}
} else if (!road[r].trafficPresent &&
road[r].sensorFault) {
road[r].sensorFault = false;
road[r].trafficOnSince = 0;
}
}

static bool allRoadsEqualTraffic() {
int cnt = 0;
int fv = -1;
for (uint8_t r = 0; r < 3; r++) {
if (!road[r].trafficPresent)
continue;
cnt++;
if (fv < 0)
fv = road[r].votes;
else if (road[r].votes != fv)
return false;
}
return (cnt > 1);
}

static int8_t chooseSmartRoad() {
uint32_t now = millis();
for (uint8_t r = 0; r < 3; r++) {
if (road[r].sensorFault)
continue;
bool anyLatch =

```

```

    road[r].pirLatch ||
    road[r].usLatch ||
    road[r].irLatch;
uint32_t redDur =
    now - road[r].redSince;
if (anyLatch &&
    redDur >= ANTI_STARVE_MS) {
    return (int8_t)r;
}
}
bool any = false;
for (uint8_t r = 0; r < 3; r++) {
    if (road[r].trafficPresent) {
        any = true;
        break;
    }
}
if (!any) return -1;
if (allRoadsEqualTraffic())
    return -1;
int8_t chosen = -1;
uint32_t maxW = 0;
for (uint8_t r = 0; r < 3; r++) {
    if (!road[r].trafficPresent ||
        road[r].sensorFault)
        continue;
    uint32_t w =
        now - road[r].redSince;
    bool better = (w > maxW);
    bool tieWin =
        (w == maxW) &&
        (chosen >= 0) &&
        (r < (uint8_t)chosen);
    if (better || tieWin) {
        maxW = w;
        chosen = (int8_t)r;
    }
}

```



```

    }
}
return chosen;
}

static void runGreenPhase(uint8_t r, bool isDefault);
static void runGreenPhase(uint8_t r, bool isDefault) {
    activeRoad = (int8_t)r;
    road[r].lastGreenAt = millis();
    uint32_t phaseStart = millis();
    uint32_t hardDeadline =
        phaseStart + GREEN_CAP_MS;
    uint32_t phaseEnd =
        phaseStart +
        (isDefault ? DEFAULT_GREEN_MS :
            MIN_GREEN_MS);
    if (phaseEnd > hardDeadline)
        phaseEnd = hardDeadline;
    bool extended = false;
    bool switchPending = false;
    int8_t pendingRoad = -1;
    writeLEDs(buildPattern(r, false));
    uint32_t lastStatusLog = 0;
    while (millis() < phaseEnd) {
        wdt_reset();
        snapshotAllSensors();
        for (uint8_t i = 0; i < 3; i++) {
            processFusion(i);
        }
        if (!isDefault &&
            !extended &&
            road[r].trafficPresent) {
            if (hardDeadline > phaseEnd) {
                phaseEnd = hardDeadline;
                extended = true;
            }
        }
    }
}

```

```

if (isDefault && !switchPending) {
    int8_t smart =
        chooseSmartRoad();
    if (smart >= 0 &&
        smart != (int8_t)r) {
        switchPending = true;
        pendingRoad = smart;
        uint32_t cut =
            millis() + 1000UL;
        if (cut < phaseEnd)
            phaseEnd = cut;
    }
}

if (millis() - lastStatusLog
    >= LOG_INTERVAL_MS) {
    lastStatusLog = millis();
    printStatusTable();
}

delay(200);
}

writeLEDs(buildPattern(r, true));
uint32_t yellEnd =
    millis() + YELLOW_MS;
while (millis() < yellEnd) {
    wdt_reset();
    delay(100);
}

setAllRed();
delay(ALL_RED_MS);
road[r].redSince = millis();
activeRoad = -1;
if (switchPending &&
    pendingRoad >= 0) {
    runGreenPhase(
        (uint8_t)pendingRoad,
        false

```

```

    );
}
}

void setup() {
    Serial.begin(9600);
    wdt_enable(WDTO_4S);
    pinMode(SR_DATA, OUTPUT);
    pinMode(SR_CLK, OUTPUT);
    pinMode(SR_LATCH, OUTPUT);
    for (uint8_t r = 0; r < 3; r++) {
        pinMode(PIR_PINS[r], INPUT);
        pinMode(TRIG_PINS[r], OUTPUT);
        pinMode(ECHO_PINS[r], INPUT);
        pinMode(IR_PINS[r], INPUT);
    }
    uint32_t now = millis();
    for (uint8_t i = 0; i < 3; i++) {
        Road& rd = road[i];
        rd.pirRaw = false;
        rd.irRaw = false;
        rd.usDist = 999;
        rd.prevUsDist = 999;
        rd.pirActiveStart = 0;
        rd.pirConfirmed = false;
        rd.pirLatchExpiry = 0;
        rd.pirLatch = false;
        rd.usActiveStart = 0;
        rd.usConfirmed = false;
        rd.usLatchExpiry = 0;
        rd.usLatch = false;
        rd.lastUsDist = 999;
        rd.staticSince = 0;
        rd.usFault = false;
        rd.irActiveStart = 0;
        rd.irConfirmed = false;
        rd.irLatchExpiry = 0;
    }
}

```

```

    rd.irLatch    = false;
    rd.votes = 0;
    rd.trafficPresent = false;
    rd.lastVoteDropTime = 0;
    rd.inHoldoff = false;
    rd.trafficOnSince = 0;
    rd.sensorFault = false;
    rd.prevTraffic = false;
    rd.prevVotes = 0;
    rd.lastGreenAt =
        now - (uint32_t)(i * 4000UL);
    rd.redSince =
        now - (uint32_t)(i * 4000UL);
}
setAllRed();
}

static uint32_t lastRawLog = 0;
void loop() {
    wdt_reset();
    snapshotAllSensors();
    for (uint8_t r = 0; r < 3; r++) {
        processFusion(r);
        wdt_reset();
    }
    if (millis() - lastRawLog
        >= LOG_INTERVAL_MS) {
        lastRawLog = millis();
        printStatusTable();
    }
    int8_t smart =
        chooseSmartRoad();
    if (smart >= 0) {
        runGreenPhase(
            (uint8_t)smart,
            false
        );
    }
}

```

```
} else {  
    runGreenPhase(  
        defaultRoad,  
        true  
    );  
    defaultRoad =  
        (defaultRoad + 1) % 3;  
}  
}
```